Programare Dinamică

Paranteziarea matricelor

T: $O(n^3)$
S: $\Theta(n^2)$

- $m[i,j] = \begin{cases} 0 & , i = j \\ m[i,k] + m[k+1,j] + d_{i-1} \cdot d_k \cdot d_j & , i < j \end{cases}$

- $A_1 \cdot A_2 \cdot A_3 \cdot A_4 \rightarrow (A_1 \cdot (A_2 \cdot A_3)) \cdot A_4 \rightarrow 158$
  $5\times4 \quad 4\times6 \quad 6\times2 \quad 2\times7$

- $m[1,1] = m[2,2] = m[3,3] = m[4,4] = 0$
- $m[1,2]$: $A_1 \cdot A_2 = 5\times4 \cdot 4\times6 = 5\cdot4\cdot6 = 120$
- $m[2,3]$: $A_2 \cdot A_3 = 4\times6 \cdot 6\times2 = 4\cdot6\cdot2 = 48$
- $m[3,4]$: $A_3 \cdot A_4 = 6\times2 \cdot 2\times7 = 6\cdot2\cdot7 = 84$

- $m[1,3] = A_1 \cdot A_2 \cdot A_3 \begin{cases} A_1 \cdot (A_2 \cdot A_3) \\ (A_1 \cdot A_2) \cdot A_3 \end{cases} = \min = 88$

$A_1 \cdot (A_2 \cdot A_3) = m[1,1] + m[2,3] + 5\times4\cdot2 = 88$
$5\times4 \quad 4\times6 \quad 6\times2$

$(A_1 \cdot A_2) \cdot A_3 = m[1,2] + m[3,3] + 5\cdot6\cdot2 = 180$
$5\times4 \quad 4\times6 \quad 6\times2$

m	1	2	3	4
1	0	120	88	158
2		0	48	104
3			0	84
4				0

s	1	2	3	4
1		1	1	3
2			2	3
3				3
4				

- $m[2,4] = A_2 \cdot A_3 \cdot A_4 \begin{cases} A_2 \cdot (A_3 \cdot A_4) \\ (A_2 \cdot A_3) \cdot A_4 \end{cases} = 104$

$A_2 \cdot (A_3 \cdot A_4) = m[2,2] + m[3,4] + 4\cdot6\cdot7 = 252$
$4\times6 \quad 6\times2 \quad 2\times7$

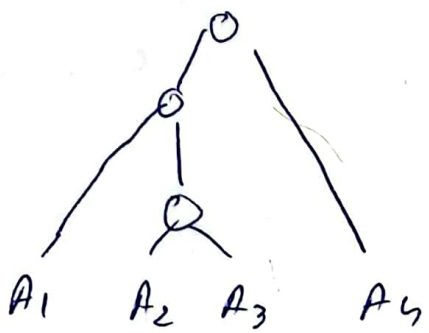
$(A_2 \cdot A_3) \cdot A_4 = m[2,3] + m[4,4] + 4\cdot2\cdot7 = 104$

- $m[1,4] = \min\{ m[1,1] + m[2,4] + 5\cdot4\cdot7,$
  $m[1,2] + m[3,4] + 5\cdot6\cdot7,$
  $m[1,3] + m[4,4] + 5\cdot2\cdot7 \}$

$= \min\{ 104 + 1\times0,$
$120 + 84 + 210$
$88 + 0 + 70 \}$

$\rightarrow (A_1 \cdot A_2 \cdot A_3) \cdot A_4$

$(A_1 \cdot (A_2 \cdot A_3)) \cdot A_4$



Greedy

Algoritmul Huffman

$T: O(n \log n)$

• Mesaj: „ce_abecedar" length = 11

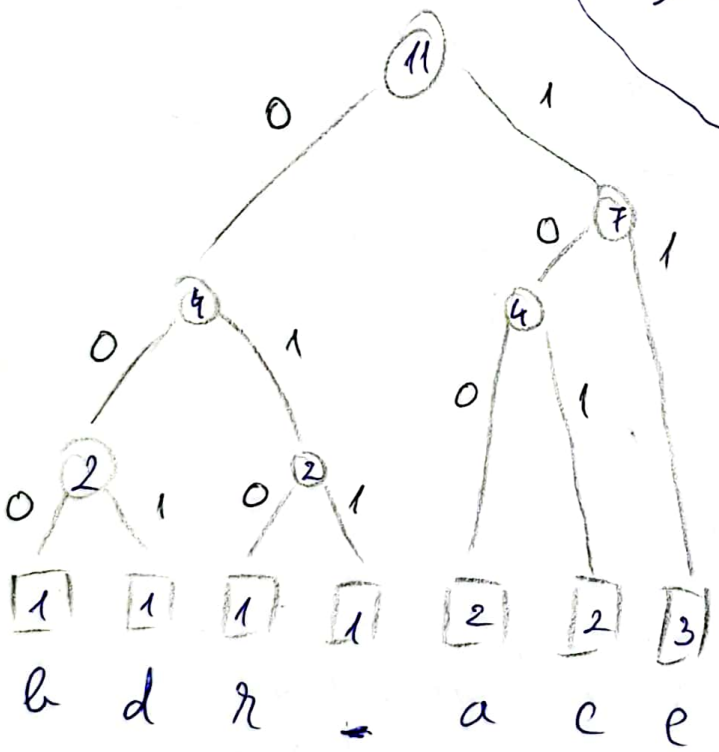
Char	Aparitii	Frecv	Cod
a	2	2/11	000
b	1	1/11	001
c	2	2/11	010
d	1	1/11	011
e	3	3/11	100
_	1	1/11	101
r	1	1/11	110

Char	Apar	Cod	Biti
a	2	100	2·3=6
b	1	000	1·3=3
c	2	101	2·3=6
d	1	001	1·3=3
e	3	11	3·2=6
r	1	010	1·3=3
_	1	011	1·3=3

30 biti

$\Sigma d_i \cdot f_i$ = Sum. muchiilor până la litera + nr. aparitii litera

Codul



↳ nr. char · 8 (7·8=56)
+ nr. total. cifre. cod (20)
+ Biti (30)
↳ 106 biti

ce_abecedar → 101 11 011 100 000 11 101 11 001 100 010
c e _ a b e c e d a r

Backtracking. PSR

- Restricție = relație între una sau mai multe variabile
- PSR (Problema de satisfacere a restricției) = un triplet $<V, D, C>$ unde - V = mulțimea de n variabile, D - domeniul de valori al variabilelor, C - mulțimea de restricții peste submulțimi ale lui V
- Soluție PSR = tuplu $(x_1, \dots, x_n)$ ce conține variabilele V, din domeniul corespunzător D, care satisfac restricțiile din C
- PSR binar = restricții unare sau binare
- Alg. rezolvare PSR - Caracteristici
  1. Rezolvă PSR binar
  2. Variabilele au domenii finite
  3. Prin propagarea restricțiilor se filtrează mulțimile de valori
  4. Procesul de propagare se oprește când
     - O mulțime de valori este vidă. ($\exists E \subset C$)
     - Nu se mai face nicio modificare asupra domeniilor vreunei mulțimi

- Alg. de consistență a nodurilor NC-1 ($O(na)$)

```
Alg NC-1                          n = nr de variabile
  Pentru i de la 1 la n           Qi - restricția unară pe variabila i
    Execută NC(i)                 Di - domeniul variabilei i
  Sfârșit

NC(i)
  Pentru fiecare x ∈ Di
    ~~Dacă nu există y ∈ Dj a.i. Pij(x,y)~~ //
    Dacă not Qi(x) //nu este satisfăcută restricția unară
      Șterge x din Di
  Sfârșit
```

- Alg de consistență a arcelor

```
REVISE((i,j))
  ȘTERS ← Fals
  Pentru fiecare x ∈ Di
    Dacă nu există y ∈ Dj a.i. Pij(x,y)
      Șterge x din Di
      Șters ← True
  Întoarce Șters

O(a²)
```

```
AC-1
  NC-1
  Q ← {(i,j) | (i,j) ∈ arce(G), i ≠ j}
  Repetă
    Schimbat ← fals
    Pt fiec. (i,j) ∈ Q
      Schimbat ← or(REVISE(i,j), Sch)
  Până când non Schimbat

O(n·a·2r·a²)
```

```
AC-3
  NC-1
  Q ...

Cât t. Q nevid
  selectează și șterge
    un arc (k,m) ∈ Q
  °Dacă revise((k,m))
    Q ← Q ∪ {(i,k) |
    (i,k) ∈ arce(G)
      i ≠ k, i ≠ m}

O(a³r)
```

- Alg de consistență a nodurilor NC-1 ($O(na)$)

NC(i)
Pt. fiecare $x \in D_i$
Dacă $\neg Q_i(x)$ // nu o satisface
șterg x din D_i — restricția unora
Sfârșit

NC-1
Pt fiecare i de la 1 la n
Execută NC(i)
Sfârșit

- Alg de consistență a arcelor

REVISE((i,j)) ($O(a^2)$)
șters ← fals
Pentru fiecare $x \in D_i$
Dacă $\nexists y \in D_j$ a.i $P_{ij}(x,y)$
șterge x din D_i
șters ← true
Întoarce șters

REVISE(i,j) ($O(a^2)$)
șters ← fals
Pentru fiecare $x \in D_i$
Dacă nu există $y \in D_j$ a.i $P_{ij}(x,y)$
Șterge x din D_i
șters ← adevărat
Întoarce șters

AC-1 ($O(n \cdot a \cdot 2r \cdot a^2)$)
NC-1
$Q \leftarrow \{(i,j) \mid (i,j) \in arce(G), i \neq j\}$
Repetă
schimbat ← fals
Pt fiecare $(i,j) \in Q$
schimbat ← (REVISE(i,j) or schimbat)
Până când non schimbat

AC-1 ($O(n \cdot a \cdot 2r \cdot a^2)$)
NC-1
$Q \leftarrow \{(i,j) \mid (i,j) \in arce(G), i \neq j\}$
Repetă
schimbat ← fals
Pentru fiecare $(i,j) \in Q$
schimbat ← (Revise((i,j)) or schimbat)
Până când non schimbat

AC-3 ($O(a^3 r)$)
NC-1
$Q \leftarrow \{(i,j) \mid (i,j) \in arce(G), i \neq j\}$
Cât timp Q nu e goală
Selectează și șterge un arc din $Q = (k,m)$
Dacă (REVISE(k,m))
$Q \leftarrow Q \cup \{(i,k) \mid (i,k) \in arce(G)$
$i \neq k, i \neq m\}$

AC-3 ($O(a^3 r)$)
NC-1
$Q \leftarrow \{(i,j) \mid (i,j) \in arce(G), i \neq j\}$
Cât timp Q nu este nulă
Selectează și șterge (k,m) din Q
Dacă (REVISE((k,m))
$Q \leftarrow \{(i,k) \mid (i,k) \in arce(G)$
$i \neq k, i \neq m\}$

Alg. de consistență a arcelor

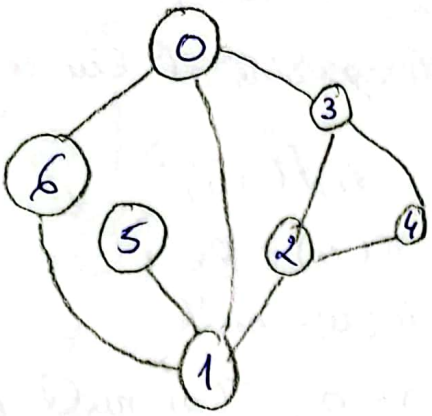
REVISE(i,j)
șters ← fals
Pt. fiecare $x \in D_i$
Dacă $\nexists y \in D_j$ a.i $P_{ij}(x,y)$
șterge x din D_i
șters ← adevărat
Întoarce - șters

AC-1
NC-1
$Q \leftarrow \{(i,j) \mid (i,j) \in arce(G), i \neq j\}$
Repetă
Schimbat ← fals
Pt fiecare (i,j) din Q
Schimbat ← {Revise(i,j), Schimbat}
Revise
Întoarce Schimbat

AC-3
NC-1
Q ....
Cât timp Q nu este nulă
Selectează și șterge (k,m) din Q
Dacă (Revise(k,m),
$Q \leftarrow Q \cup \{(i,k) \mid$
$(i,k) \in arce(G)$
$i \neq k, i \neq m\}$

BFS $O(n+m)$

- Cerința



- Sursa 0
- Răspuns: 0, 6, 1, 3, 5, 2, 4

1. - Q = 0, d(0) = 0, p(0) = null
   - adaug în coadă vecinii lui 0 : 6, 1, 3 cu culoarea gri
   - scot 0 din coadă, col(0) = negru
2. Q : 6, 1, 3 ; d(6) = d(1) = d(3) = d(0) + 1 = 1, p(6) = p(1) = p(3) = 0
   - adaug în coadă copiii lui 6 : 1 are culoarea gri => nu îl mai adug
   - scot 6 din coadă, col(6) = negru
3. Q = 1, 3
   - adaug în coadă copiii lui 1 : 5, 2 (6 și 0 au culoarea negru) gri
   - scot 1 din coadă, col(1) = negru
4. Q = 3, 5, 2 d(3) = 1 d(5) = d(2) = d(1) + 1 = 2, p(3) = 0 p(5) = p(2) = 1
   - adaug în coadă copiii lui 3 cu gri : 4 (2 este negri)
   - d(4) = d(3) + 1 = 2 p(2) = p(4) = 3
   - scot 3 din coadă, col(3) = negru
5. Q = 5, 2, 4
   - adaug în coadă copiii albi ai lui 5 (nu există)
   - scot 5 din coadă, col(5) = negru
6. Q = 2, 4
   - adaug în coadă copiii albi ai lui 2 (nu există)
   - scot 2 din coadă
7. Q = 4
   - adaug în coadă copiii albi ai lui 4 (nu există)
   - scot 4 din coadă.
8. Q = mulțime vidă => am terminat parcurgerea

Corectitudinea BFS

- BFS este corect și după terminarea $\delta(s,u) = dist(s,u), \forall u \in V$
- Fie $V_k = \{u \in V \mid \delta(s,u) = k\}$, $d(u)$ = timpul de descoperire al lui u
- Demonstrez prin inducție

$P(k) = \forall u \in V_k$ cu proprietățile $\begin{cases} \delta(s,u) = dist(s,u) \\ k>0 \Rightarrow \pi(u) \in V_{k-1} \\ k=0 \Rightarrow \pi(u) = null \end{cases}$

- Caz de bază: $k=0$, $V_0 = \{s\}$, $\delta(s,s) = dist(s,s) = 0$, $\pi(s) = null$ A
- Pas de inducție $k>0$

$P(k)$ A $\Rightarrow P(k+1)$ A.

- Fie la întâmplare $v \in V_{k+1}$, $\delta(s,v) = k+1$

$u \in V_k$ predecesorul lui v pe drumul $s..v$

- Sunt 3 cazuri

a) u este descoperit după v

$dist(s,v) \leq dist(s,u)$
$dist(s,v) = k+1$
$dist(s,u) = k$
$\Rightarrow k+1 \leq k$ - imposibil

b) u este descoperit înainte de v, iar v este descoperit pe arcul (u,v)

$dist(s,v) = dist(s,u) + 1$
$\pi(v) = u$
$\Rightarrow P(k+1)$ - adevărată

c) u este descoperit înainte de v

și fie $z \neq u$ a.î. v este descoperit pe arcul (z,v)

- $d(z) < d(z) < d(u) \Rightarrow$ culoarea lui u = alb la $d(u)$

$\Rightarrow v$ descoperit pe arcul (u,z) (contradicție cu ipoteza

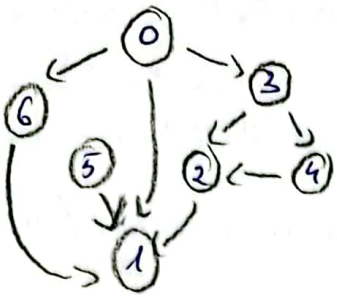
- $d(z) < d(u) \Rightarrow dist(s,z) \leq dist(s,u) \Rightarrow dist(s,z) \leq k$

$dist(s,v) = dist(s,z) + 1$
$dist(s,v) = k+1$
$\Rightarrow dist(s,z) = k$

$\Rightarrow$ drumul $s..z, v$ este optim $\Rightarrow z \in V_k$
$\pi(v) = z$
$\Rightarrow P(k+1)$ A.

Adâncime

DFS $O(n+m)$

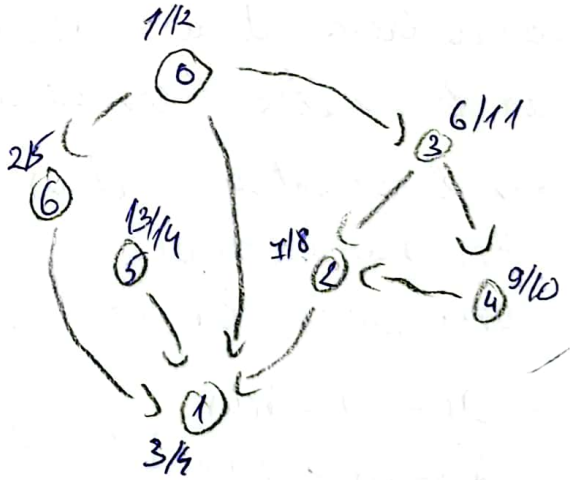


- Nu avem nod sursă
- Se incepe, pe rând, din fiecare nod, dacă nodul nu a fost deja vizitat (este alb)
- Pentru fiecare nod u
  c(u) = alb π(u) = null
- t = 0 d(u) - timp de descoperire u f(u) - timp de finalizare u

1. 0 - alb → explorează(0)
   d(0) = 1, t = 1
   c(0) = gri
   succ_albi(0) = {6, 1, 3}
   π(6) = π(1) = π(3) = 0
2. 6 - alb → exp(6)
   d(6) = 2, t = 2
   c(6) = gri
   succ_albi(6) = {1}
   π(1) = 6
3. 1 - alb → exp(1)
   d(1) = 3, t = 3
   c(1) = gri
   succ_albi(1) = _
   c(1) = negru
   f(1) = 4, t = 4
4. succ_albi(6) = _
   c(6) = negru
   f(6) = 5, t = 5
5. succ_albi(0) = {3}
   π(3) = 0
6. 3 alb → exp(3)
   d(3) = 6, t = 6
   c(3) = gri
   succ_albi(3) = {2, 4}
   π(2) = π(4) = 3
7. 2 - alb - exp(2)
   d(2) = 7, t = 7
   c(2) = gri
   succ_albi(2) = _
   f(2) = 8, t = 8
   c(2) = negru
8. succ_albi(3) = {4}
   π(4) = 3
9. 4 - alb → exp(4)
   d(4) = 9, t = 9
   c(4) = gri
   succ_albi(4) = _
   f(4) = 10, t = 10
   c(4) = negru
10. succ_albi(3) = _
    f(3) = 11, t = 11
    c(3) = negru
    π(3) = 0
11. succ_albi(0) = _
    f(0) = 12, t = 12
    c(0) = negru
    π(0) = null
    albe = {5}
12. explorează(5)
    d(5) = 13, t = 13
    c(5) = gri
    succ_albi(5) = _
    f(5) = 14, t = 14
    c(5) = negru

albe = {_}

s-a terminat parcurgerea



Teorema parantezelor

- $\forall u, v$ avem $I(u) \cap I(v) = \emptyset$ sau $I(u) \subset I(v)$ sau $I(v) \subset I(u)$
- Demonstrație prin considerarea tuturor combinațiilor posibile

a) $I(u) < I(v)$: $d(u) < f(u) < d(v) < f(v)$: $v \notin R(u)$.
=> v rămâne alb pe durata prelucrării lui u => $f(u) < d(v)$

b) $I(v) \subset I(u)$: $d(u) < d(v) < f(v) < f(u)$: $v \in R(u)$
=> v este descoperit și își devine negru înainte terminării lui u => $f(u) < f(v)$

c) $I(v) < I(u)$ analog a d) $I(u) \subset I(v)$ analog b

Teorema drumurilor albe

- $G = (V, E)$, $Arb(u)$

v - descendent al lui u în $Arb(u)$ <=> la $d(u)$ ∃ o cale numai cu noduri albe $u \dots v$

- Demonstrația se face prin inducție

a) $v \in V(u)$ => la $d(u)$ ∃ o cale numai cu noduri albe $u \dots v$
- $v \in V(u)$ => ∃ $u \dots z \dots v$
- Fie z din α => $d(u) < d(z) < f(z) < f(u)$ => la $d(u)$, $c(z) =$ alb
- la $d(u)$, $c(z) =$ alb
z - la întâmplare => toate nodurile de pe calea α sunt albe

b) la $d(u)$ ∃ o cale numai cu noduri albe $u \dots v$ => $v \in V(u)$
- Fie $u, v_0, v_1, \dots, v_n = v$ o cale a.î. la $d(u)$, $c(v_i) =$ alb, $\forall i \in \overline{0, n}$
- Demonstrez că v_i este descendentul lui u în $Arb(u)$
- Caz de bază: $d(u) < d(v_i) < f(u)$ => v_i descendentul lui u A
- Pas de inducție : v_i descendent al lui u => v_{i+1} descendent al lui u

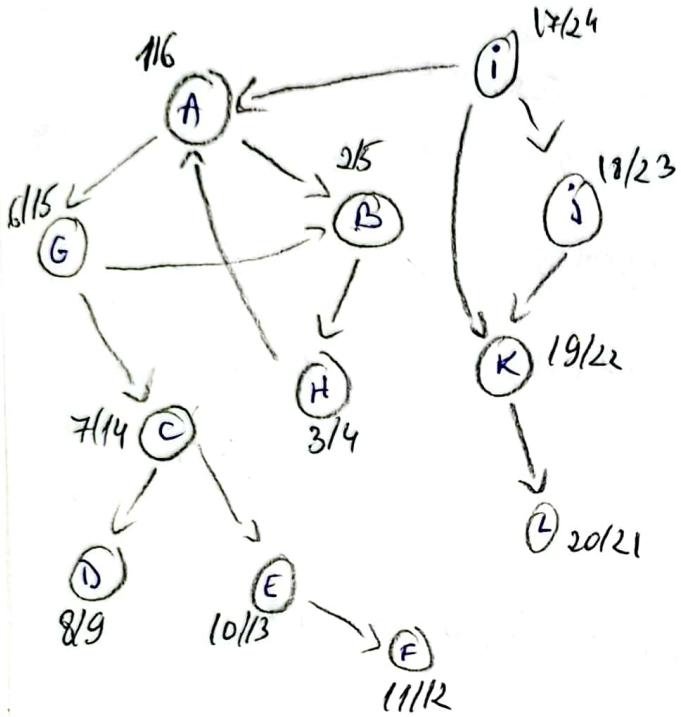
- v_i descendent u => $d(u) < d(v_i) < f(v_i) < f(u)$
- v_{i+1} alb la $d(u)$
v_{i+1} succesor lui v_i => v_{i+1} descoperit după $d(u)$
$d(v_{i+1}) > d(u)$
- $d(v_{i+1}) > d(u)$
$d(v_{i+1}) < f(v_i)$ => $d(u) < d(v_{i+1}) < f(v_i) < f(u)$

=> v_{i+1} descendent al lui u

=> ~~v_{i+1}~~

Clasificări ale arcelor grafului

- Arc (u, v)
  - direct gri -> alb
  - înainte gri -> negru $d(gri) < d(negru)$
  - invers gri -> gri
  - transversal gri -> negru $d(gri) > d(negru)$



- arc direct (de arbore)
  AB, BH, AG, GC, CD, CE, EF, iJ, JK, KL
- arc invers (de ciclu)
  HA
- arc înainte
  iK
- arc transversal
  GB, iA

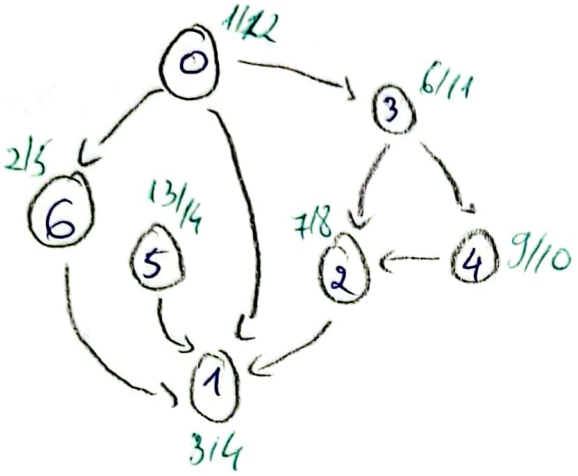
Sortare topologică $O(n+m)$

" Graf orientat și aciclic

- Idee bazată pe DFS

- la sfârșitul lui DFS avem $\forall (u,v) \in E, f(v) < f(u)$

-> x pun în V_S vârfurile în ordine descrescătoare a timpilor f



- La sfârșitul lui DFS

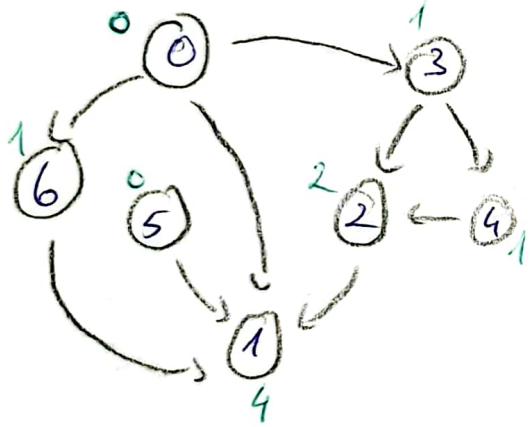
1 6 2 4 3 0 5

- Sortare topologică:

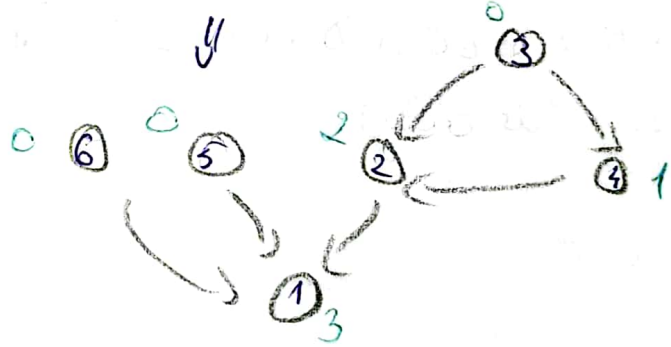
$V_S = \{5, 0, 3, 4, 2, 6, 1\}$

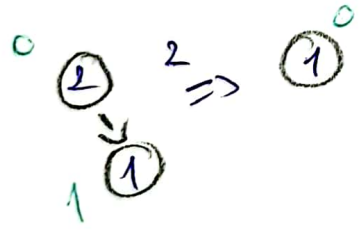
- Idee bazată pe grad interior

$V_S = \{0, 6, 5, 3, 4, 2, 1\}$

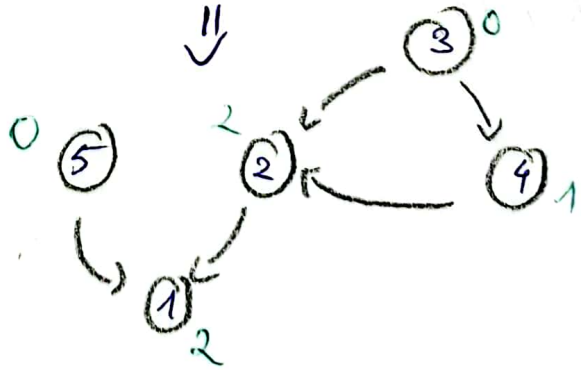


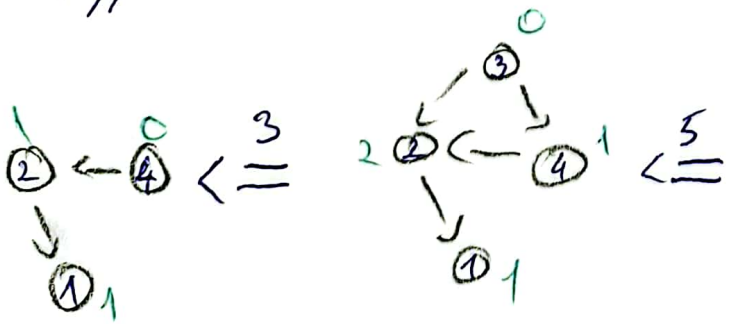
"0" are grad interior 0 => îl elimin





"6" are grad interior 0 => îl elimin



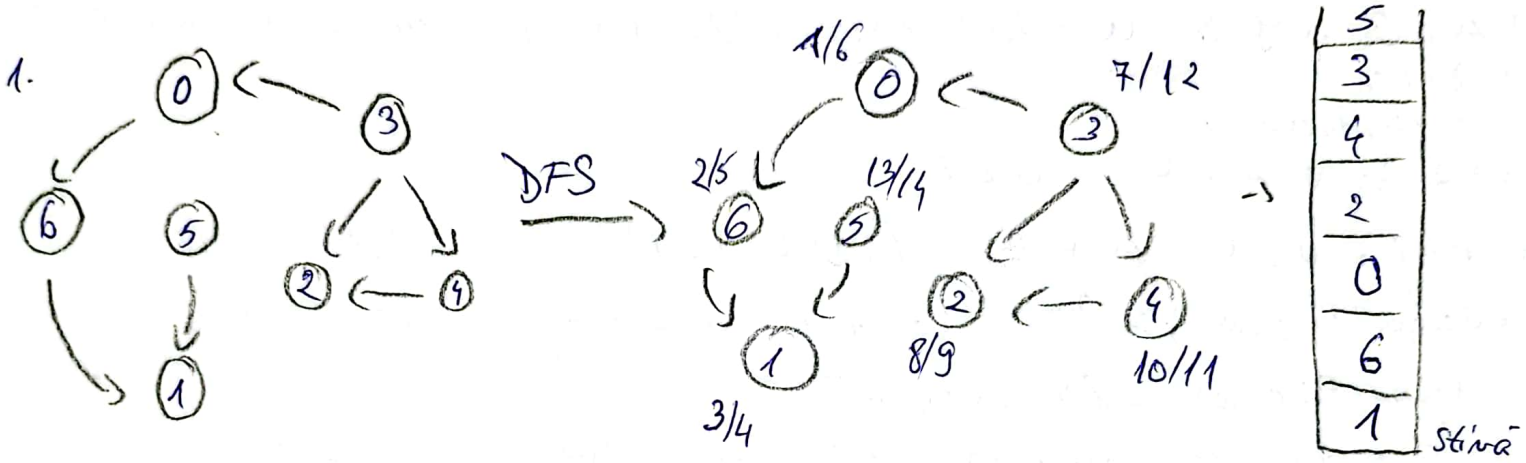


Algoritmul lui Kosaraju pentru găsirea componentelor tari conexe CTC

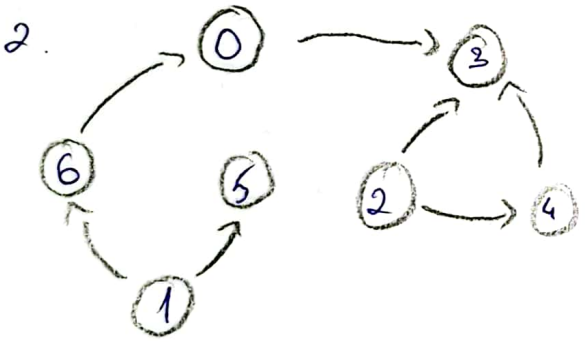
$O(n+m)$

1. Aplic DFS și pun pe o stivă rezultatul (primul nod care devine negru este la baza stivei)
2. Inversez arcele pentru a obține graful transpus.
3. Scot câte un element din stivă și aplic DFS din nodul indicat de element. După ce se termină DFS => am găsit o CTC.
4. Repet pasul 3 având grijă să nu iau ca nod sursă un nod vizitat.

1.



2.



1.5

3-4 : ~~1~~ 3 (DFS - nu are vecini nevizitați)

3. 4

~~4~~. 2

~~5~~. 0

6. 6

7. 1

Corectitudine

găsiți de DFS(G^T)

- Dem. prin inducție că vârfurile din fiecare arbore formează o CTC
- La fiecare pas îl demonstrează că arborele din acel pas e CTC
- $P_n \Rightarrow P_{n+1}$: T – arbore curent, $C_r = \{v \in V \mid \phi(v) = r\}$
  r – rădăcina
  strămoș

- Dem. $u \in T \iff u \in C_r$

a) $u \in C_r \Rightarrow u \in T$

1. $u \in C_r \Rightarrow \exists\, r \rightsquigarrow u \Rightarrow$ toate nodurile din C_r ajung în același arbore
2. $r \in C_r$
   r – rădăcină T

1 + 2 $\Rightarrow (\forall)\, u \in C_r \Rightarrow u \in T$

b) $u \in T \Rightarrow u \in C_r \iff \forall w$ a.î. $f(\phi(w)) \geq f(r)$ sau $f(\phi(w)) < f(r)$

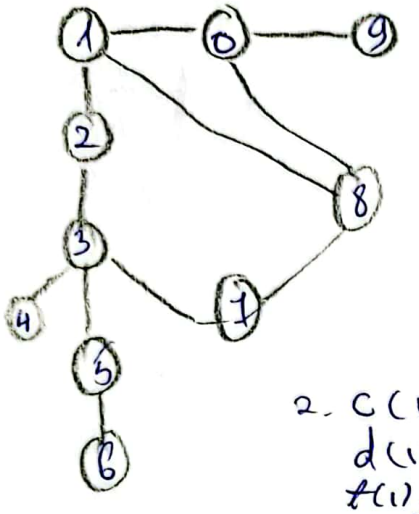
- dacă $f(\phi(w)) \geq f(r) \Rightarrow$ la $d(r)$, w este deja în CTC $\Rightarrow w \notin T$
- dacă $f(\phi(w)) < f(r) \Rightarrow w \notin T$

$\Rightarrow T$ conține doar nodurile pentru care $\phi(v) = r \Rightarrow T = C_r$

Tarjan. $O(n+m)$

Puncte de articulatie

- low(u) = nodul vizitat cel mai devreme ce poate fi atins prin subarborele DFS cu radacina u



- Initial $v=0$
  $c(v)$: alb, $d(v) = low(v) = 0$, $\pi(v) = null$
  $subarb(v) = art(v) = timp(v) = 0$.

1. $v=0 \rightarrow c(0) = gri$
   $d(0) = low(0) = 0$
   $t(0) = 1$ $subarb(0) = 1$ $expl(1)$

2. $c(1) = gri$
   $d(1) = low(1) = 1$
   $t(1) = 2$ $subarb(1) = 1$ $\pi(1) = 0$
   $expl(2)$

3. $c(2) = gri$
   $d(2) = low(2) = 2$
   $t(2) = 3$ $subarb(2) = 1$ $\pi(2) = 1$
   $expl(3)$

4. $c(3) = gri$
   $d(3) = low(3) = 3$
   $t(3) = 4$ $subarb(3) = 1$
   $expl(4)$

5. $c(4) = gri$
   $d(4) = low(4) = 4$
   $t(4) = 5$ $subarb(4) = 0$
   revenire
   $c(4) = negru$

6. $low(3) = min(low(4), low(3))$
   $low(3) = 3$
   $subarb(3) = 2$
   $expl(5)$

7. $c(5) = gri$
   $d(5) = low(5) = 5$
   $t(5) = 6$ $subarb(5) = 1$
   $expl(6)$

8. $c(6) = gri$
   $d(6) = low(6) = 6$
   $t(6) = 7$ $subarb(6) = 0$
   revenire
   $c(6) = negru$

9. $low(5) = min(low(5), low(6))$
   $low(5) = 5$
   revenire
   $c(5) = negru$

10. $low(3) = min(low(3), low(5))$
    $low(3) = 3$
    $subarb(3) = 3$
    $expl(7)$

11. $c(7) = gri$
    $d(7) = low(7) = 7$
    $t(7) = 8$ $subarb(7) = 1$
    $expl(8)$

12. $c(8) = gri$
    $d(8) = low(8) = 8$
    $t(8) = 9$
    $low(8) = min(low(8), d(0)) = 0$
    $low(0) = min(low(0), d(1)) = 0$
    revenire
    $c(8) = negru$

13. $low(7) = min(low(7), low(8))$
    $= 0$
    $low(8) < d(7)$
    $\Rightarrow art(7) = 0$
    revenire
    $c(7) = negru$

14. $low(3) = 0$
    $low(7) < d(3)$
    $\Rightarrow art(3) = 0$
    revenire
    $c(3) = negru$

15. $low(2) = 0$
    $low(3) < d(2)$
    $\Rightarrow art(2) = 0$
    $c(2) = negru$
    revenire.

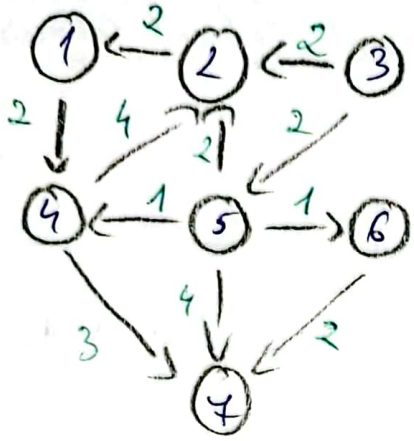
$low(1) = 0$

$c(9) = gri$
$low(9) = d(9) = 10$
$t(9) = 11.$
$low(0) = 0$
$low(9) < d(0)$
$\Rightarrow art(0) = 1$

Dijkstra

Vectori $O(V^2)$
HB $O(E \log V)$
HF $O(V \log V + E)$

! doar pentru costuri pozitive



Inițial

$Q = \{\}$

$d[1] = d[2] = d[3] = d[4] = d[5] = d[6] = \infty = d[7]$

π = null

- Dacă $d[succesor] > d[curent] + w(curent, succesor)$
  $\Rightarrow d[succesor] = d[c] + w \dots$

1. $s = 5$
   $d[5] = 0$
   $Q = \{5\}$

2. $u = 5$
   $succ[5] = \{4, 2, 6, 7\}$
   $d[4] > d[5] + w(5,4) \Leftrightarrow \infty > 1 \Rightarrow d[4] = 1$
   $d[2] > d[5] + w(5,2) \Leftrightarrow \infty > 2 \Rightarrow d[2] = 2$
   $d[6] > d[5] + w(5,6) \Leftrightarrow \infty > 1 \Rightarrow d[6] = 1$
   $d[7] > d[5] + w(5,7) \Leftrightarrow \infty > 4 \Rightarrow d[7] = 4$
   $Q = \{4, 2, 6, 7\}$

3. $u = 4$
   $succ[4] = \{7\}$ 2
   $d[7] > d[4] + w(4,7) \Leftrightarrow 4 > 4$ F.
   $d[2] > d[4] + w(4,2) \Leftrightarrow 2 > 5$ F
   $Q = \{2, 6, 7\}$

4. $u = 2$
   $succ[2] = \{1\}$
   $d[1] > d[2] + w(2,1) \Leftrightarrow \infty > 4 \Rightarrow d[1] = 4$
   $Q = \{6, 7, 1\}$

5. $u = 6$ $d[6] = 1$
   $succ[6] = \{7\}$
   $d[7] > d[6] + w(6,7) \Leftrightarrow 4 > 3$ A
   $\Rightarrow d[7] = 3$
   $Q = \{7, 1\}$

6. $u = 7$ $d[7] = 3$
   $succ[7] = \{\}$
   $Q = \{1\}$

7. $u = 1$ $d[1] = 4$
   $succ[1] = \{4\}$
   $d[4] > d[1] + w(1,4) \Leftrightarrow 1 > 6$ F.
   $Q = \{_\}$

⇓

v	1	2	3	4	5	6	7
d	4	2	INF	1	0	1	3

- Implementare cu
  - vectori pentru grafuri "dese" (multe arce)
  - heap pentru grafuri "rare" (mai multe noduri)
- Heapul Fibonacci este mai eficient decat heapul liniar, dar mai dificil de implementat.

Corectitudine Dijkstra

- Prin reducere la absurd
- Dem că la scoaterea din Q a fiecărui nod v avem $d[v] = \delta(s,v)$
- Pp. u a.i. $d[u] \neq \delta(s,u)$ la scoaterea din Q $\Rightarrow$
  $u \neq s$ și $u \in R(s)$

① $\Rightarrow$ la scoaterea lui u din Q $\exists$ s..u

② Fie p drumul optim s..u, p = s..xy..u, $x \notin Q$, $y \in Q$

- u e primul nod a.î. $d[u] \neq \delta(s,u) \Rightarrow d[x] = \delta(s,x)$ la momentul extragerii lui u din Q
  $\Rightarrow d[y] = \delta(s,y)$ prin relaxarea (x,y)
- y precede u pe drumul $p \Rightarrow d[y] = \delta(s,y) \leq \delta(s,u) \leq d[u]$
- $y \in Q$ la momentul scoaterii lui u din Q $\Rightarrow d[u] \leq d[y]$

$\Rightarrow d[y] = \delta(s,y) = \delta(s,u) = d[u]$

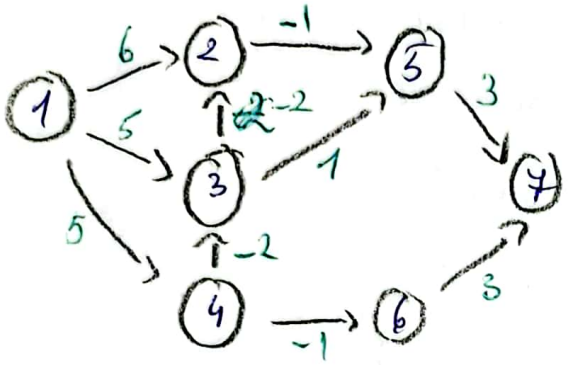
↯ Se contrazice ipoteza

$\Downarrow$

$d[u] = \delta(s,u)$

Bellman - Ford $O(N \cdot M)$

- costul minim între sursă și oricare nod
- funcționează și pentru grafuri cu costuri negative
- pentru a verifica ce muchii/arce se relaxează
  - if $d[u] + c(u,v) < d[v] \Rightarrow d[v] = d[u] + c(u,v)$
- detectează (afirmă/infirmă) că ai cicluri negative
  - Pt. fiecare (u,v), dacă $d[v] > d[u] + w(u,v) \Rightarrow$ ciclu negativ



- 7 noduri => 6 iterații
- Inițial, distanța de la sursă la fiecare nod este ∞
- Sursa este nodul 1
- $d[1] = 0$
- Pt. fiecare nod, parcurg nodurile vecine (out) și calculez $d[current] + cost(current, vecin)$

I

n	1	2	3	4	5	6	7
d	0	~~∞~~ 6	~~∞~~ 5	~~∞~~ 3	~~∞~~	~~∞~~	~~∞~~
					5	4	7

Relax:
(1,2), (1,3), (1,4), (2,5), (3,2), (4,3)
(4,6), (5,7), (6,7)

II Relax: (2,5), (3,2), (5,7)

n	1	2	3	4	5	6	7
d	0	~~6~~	3	5	~~5~~	4	~~7~~
		1			2		5

III Relax: (2,5), (5,7)

n	1	2	3	4	5	6	7
d	0	1	3	5	~~2~~	4	~~5~~
					0		3

IV Relax:

n	1	2	3	4	5	6	7
d	0	1	3	5	0	4	3

Nu se mai relaxează nicio muchie

⇓

distanțe finale

Corectitudine Bellman - Ford

- $G=(V,E)$,
  $w: E \to \mathbb{R}$ funcție de cost asociată

a) Dacă G nu conține ciclu de cost negativ $\Rightarrow$ după $|V|-1$ iterații avem $d[v] = \delta(s,v)$, $(\forall)\ v \in R(s)$

- Dem. prin inducție
- Fie $s = v_0, v_1, \dots v_k$ = drum minim în graf, $k \leq |V|-1$
- Dem că în pasul i $d[v_i] = \delta(s, v_i)$ și se menține
- P_0: $d[s] = d[v_0] = 0 = \delta(s,s) = \delta(s, v_0)$ și se menține
- $P_{i-1} \Rightarrow P_i$:
  - P_{i-1}: $d[v_{i-1}] = \delta(s, v_{i-1})$
  - în pasul i se relaxează $(v_{i-1}, v_i) \Rightarrow d[v_i] = d[v_{i-1}] + w(v_{i-1}, v_i)$
    $= \delta(s, v_{i-1}) + w(v_{i-1}, v_i)$
    $= \delta(s, v_i)$
  - $i \in (1, |V|-1) \Rightarrow$ relație adv. $(\forall)$ nod accesibil din s
    $\Rightarrow d[v] = \delta(s,v)$, $(\forall)\ v \in R(s)$

b) Dacă G conține de puțin un ciclu negativ accesibil din s
$\Rightarrow$ alg. întoarce eroare

- ∄ ciclu negativ:
  - $d[v] = \delta(s,v)$ $(\forall)\ v \in R(s)$
  - $d[v] = \delta(s,v) = \infty$, $(\forall)\ v \notin R(s)$
  - $\Rightarrow d[v] \leq d[u] + w(u,v)$ $\Rightarrow$ nu dă eroare
- ∃ ciclu negativ
  - în cel $|V|-1$ pași se scad costurile drumurilor, iar în final ciclul se menține $\Rightarrow$ Eroare.

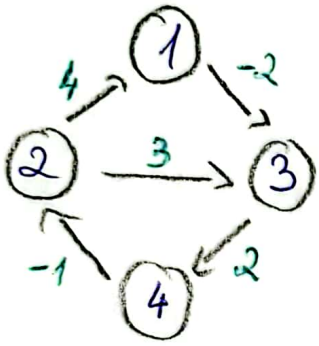
Floyd-Warshall (Roy-Floyd)

T: $O(V^3)$
S: $O(V^2)$

- distanțele minime între oricare 2 noduri = drumuri multiplicate
- exemplu de programare dinamică
- fără cicluri negative - optim pentru grafuri dense

a) Inițializări
```
i de la 1 la n
  j de la 1 la n
    d(i,j) = w(i,j)
    Dacă (w(i,j) == ∞)
      p(i,j) = null
    Altfel p(i,j) = i
```

b) Algoritm
```
k de la 1 la n
  i de la 1 la n
    j de la 1 la n
      Dacă ( d(i,j) > d(i,k) + d(k,j) )
        d(i,j) = d(i,k) + d(k,j)
        p(i,j) = p(k,j)
```



a) Inițializări

d

	1	2	3	4
1	0	∞	-2	∞
2	4	0	3	∞
3	∞	∞	0	2
4	∞	-1	∞	0

p

	1	2	3	4
1	null	null	1	-
2	2	-	2	-
3	-	-	-	3
4	-	4	-	-

b) k=1

- i=1, j=1
  d(1,1) > d(1,1)+d(1,1) F
- i=1, j=2
  d(1,2) > d(1,1)+d(1,2) F
- i=1, j=3
  d(1,3) > d(1,1)+d(1,3) F
- i=1, j=4
  d(1,4) > d(1,1)+d(1,4) F
- i=2, j=1
  d(2,1) > d(2,1)+d(1,1) F
- i=2, j=2
  d(2,2) > d(2,1)+d(1,2) F
- i=2, j=3
  d(2,3) > d(2,1)+d(1,3) A
  => d(2,3) = 4-2 = 2
  p(2,3) = p(1,3) = 1
- i=2, j=4
  d(2,4) > d(2,1)+d(1,4) F

d

	1	2	3	4
1	0	∞	-2	∞
2	4	0	2	∞
3	∞	∞	0	2
4	∞	-1	∞	0

p

	1	2	3	4
1	-	-	1	-
2	2	-	1	-
3	-	-	-	3
4	-	4	-	-

Închidere tranzitivă

$G^* = (V, E^*)$, $E^*(i,j) = \begin{cases} 1, & \exists\ i \dots j \\ 0, & \nexists\ i \dots j \end{cases}$

- prin modificarea alg: min -> sau ($\vee$)
  + -> și ($\wedge$)

Teorema Floyd-Warshall

- Fie $d^0(i,j) = w(i,j)$

  $d^k(i,j) = \min\{ d^{k-1}(i,j), d^{k-1}(i,k) + d^{k-1}(k,j) \}$

  $\Rightarrow d^n(i,j) = \delta(i,j), \forall\ i,j \in V$

a) Dem. $d^k(i,j) = \delta^k(i,j)$

- $k=0 \Rightarrow i \dots j = (i,j) \Leftrightarrow d^0(i,j) = w(i,j)$
- $0 < k \leq n$: $d^{k-1}(i,j) = \delta^{k-1}(i,j) \Rightarrow d^k(i,j) = \delta^k(i,j)$

  a) $k \notin i \dots j \Rightarrow$ drumul optim nu se modifică

  b) $k \in i \dots j \Rightarrow i \dots j$ se descompune în $i \dots k$ și $k \dots j$
- $i \dots j$ optim $\Rightarrow \delta^k(i,j) \leq \delta^{k-1}(i,j)$
- $d^k(i,j) = \min\{ d^{k-1}(i,j), d^{k-1}(i,k) + d^{k-1}(k,j) \} = \delta^k(i,j)$

b). $k = n$, $i \dots j$ trece prin $\forall\ v \in V \Rightarrow d^k(i,j) \leq d^{k-1}(i,j)$,

$\forall\ k = \overline{1,n} \Rightarrow d^n(i,j) \leq d^{k-1}(i,j), \forall\ k = \overline{1,n}$.

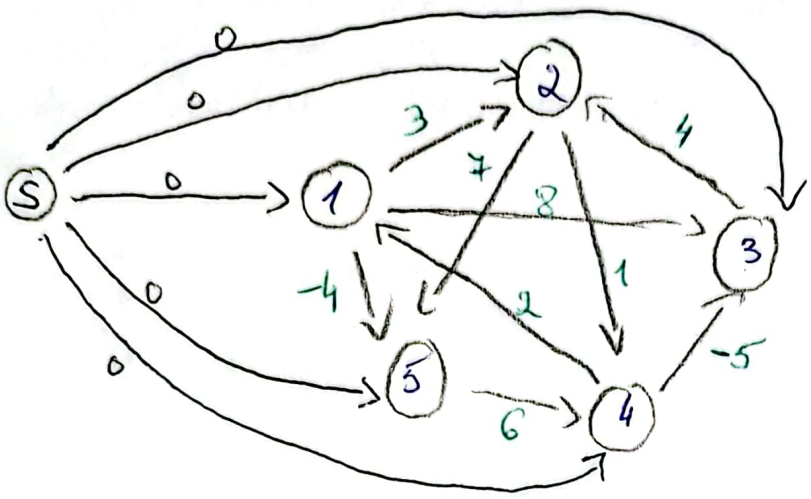
- Din a $\Rightarrow d^n(i,j) = \delta^n(i,j) \leq d^{k-1}(i,j) = \delta^{k-1}(i,j), \forall\ k = \overline{1,n}$

  $\Rightarrow d^n(i,j) = \delta^n(i,j) = \delta(i,j)$

Johnson

$O(V^2 \log V + VE)$

- drumuri de minime între oricare 2 noduri din graf
- grafuri cu costuri negative
- bun pentru grafuri rare



$w'(u,v) = w(u,v) + d[u] - d[v]$

- distanțele minime de la S la fiecare nod cu BF

$d[1] = 0 \quad d[2] = -1 \quad d[3] = -5 \quad d[4] = 0 \quad d[5] = -4$

- $w'(u,v) = w(u,v) + d[u] - d[v]$

Arce: (1,2); (1,3); (1,5); (2,5); (2,4); (3,2); (4,3); (5,4); (4,1)

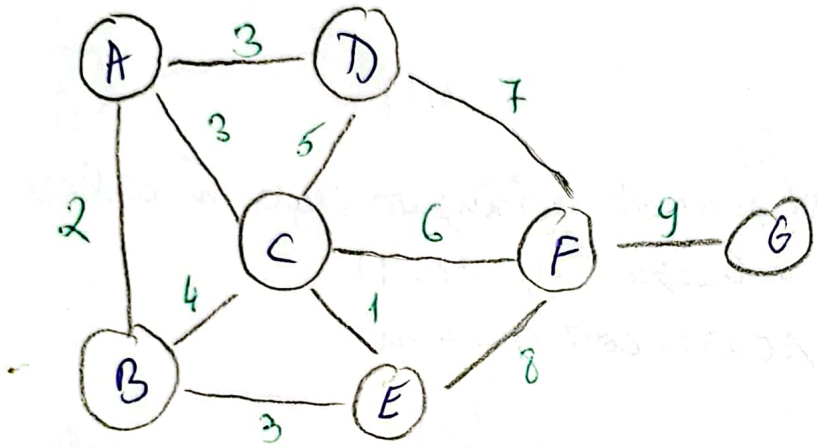
- $w'(1,2) = w(1,2) + d[1] - d[2] = 3 + 0 - (-1) = 4$
- $w'(1,3) = w(1,3) + d[1] - d[3] = 8 + 0 + 5 = 13$
- $w'(1,5) = w(1,5) + d[1] - d[5] = -4 + 0 + 4 = 0$
- $w'(2,5) = w(2,5) + d[2] - d[5] = 7 + 1 + 4 = 10$
- $w'(2,4) = w(2,4) + d[2] - d[4] = 1 - 1 - 0 = 0$
- $w'(3,2) = w(3,2) + d[3] - d[2] = 4 - 5 + 1 = 0$
- $w'(4,3) = w(4,3) + d[4] - d[3] = -5 + 0 + 5 = 0$
- $w'(5,4) = w(5,4) + d[5] - d[4] = 6 + 4 - 0 = 2$
- $w'(4,1) = w(4,1) + d[4] - d[1] = 2 + 0 - 0 = 2$

- Dijkstra cu noile costuri (fără nodul S)

Algoritmul lui Prim

Matrice : $O(V^2)$
Heap binar $O(E \log V)$
Heap Fibonacci $O(V \log V + E)$

- pentru găsirea arborilor minimi de acoperire
- se pornește cu un nod și se extinde pe rând cu muchiile cele mai ieftine care au un singur capăt în mulțimea de muchii deja formată.
- dese : matrice rare : heapuri



- Din A

nu este vecin
1

Q : B(2), C(3), D(3), E(∞), F(∞), G(∞)

→ B (actualizăm având ca nod părinte B) (A, B)

- Q : C(3), D(3), E(3), F(∞), G(∞)

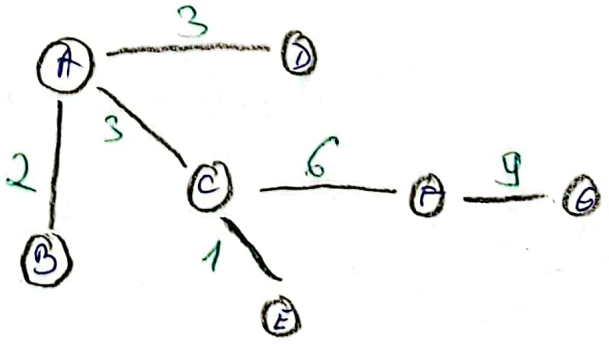
→ C (A, C)

- Q : E(1), D(3), F(6), G(∞)

→ E (C, E)

- Q : D(3), F(6), G(∞)

→ D (A, D)



- Q : F(6), G(∞)

→ F (C, F)

- Q : G(9)

→ G (F, G)

Corectitudine

1. Arătăm că muchiile care le adăugăm aparțin Arb.

- Dem. prin inducție după muchiile adăugate în AMA
- P_1: $V' = \{s\}$, $E' = \emptyset$

  adaug (u, s), u - nod adiacent lui s, aflat cel mai aproape
  $\Rightarrow (u, s) \in Arb$

- $P_n \rightarrow P_{n+1}$:
  - $S = (V', E')$ - mulțimea vf și muchii adăugate deja în arbore înainte de a adăuga $(u, \lambda[u])$
  - $\lambda[u] \in V'$, $v \notin V'$, $(u, \lambda[u])$ - cost minim
  $\Rightarrow (u, \lambda[u]) \in Arb$.

2. Arătăm că muchiile ignorate nu fac parte din ~~graf~~ Arb

- P_1. (u, v) a.i $Arb(u) = Arb(v)$

$\Rightarrow (u, v)$ creează ciclu în $Arb(u)$

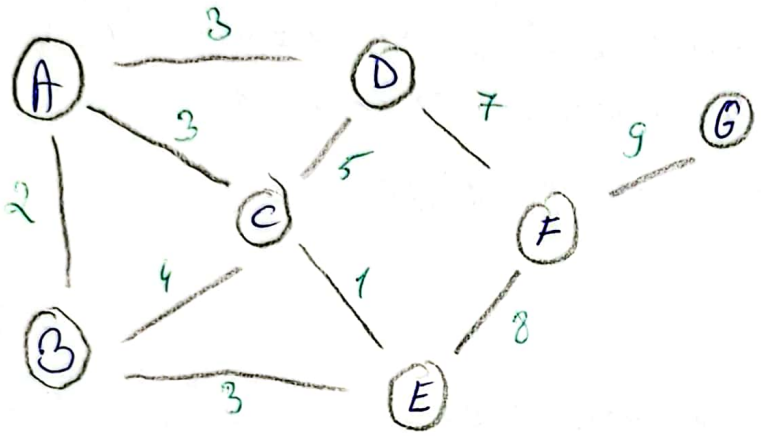
$w(u, v) = \max \{ w(u', v') \mid (u', v') \in Arb(u) \}$

$\Rightarrow (u, v) \notin Arb$

Algoritmul lui Kruskal

$O(E \log V)$

- depinde de implementarea mulțimilor disjuncte
- rezolvare prin reuniune de mulțimi
- toate nodurile formează câte o mulțime și la fiecare pas se reunesc 2 mulțimi printr-o muchie; muchiile sunt considerate în ordinea costurilor și sunt adăugate în arbore doar dacă nu e ciclu
- se sortează muchiile în funcție de cost și se alege muchia cu costul minim, dacă nodurile nu sunt deja în arbore



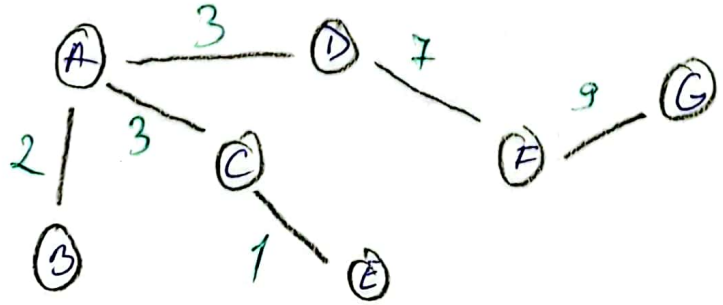
- CE=1
  AB=2
  AD=3
  AC=3
  BE=3
  BC=4
  CD=5
  DF=7
  EF=8
  FG=9

(C,E) => Aleg AB $\xrightarrow{(A,B)}$ Aleg (A, D) ⟶

Aleg (A, C) -> Nu aleg (B,E) pentru că nodurile B și E sunt deja în arborele minim de acoperire (AMA)

-> Nu aleg (B,F) -> Nu aleg (C,D) -> Aleg (D, F)

-> Nu aleg (E, F) -> Aleg (F, G)



Corectitudine

1. Arătăm că muchiile ignorate nu fac parte din Arb

- P_k: (u, v) a.i $Arb(u) = Arb(v)$

  $\Rightarrow (u, v)$ creează un ciclu în $Arb(u)$

  - $w(u, v) = \max\{w(u', v') \mid (u', v') \in Arb(u)\}$

  $\Rightarrow (u, v) \notin Arb$

2. Arătăm că muchiile pe care le adăugăm aparțin Arb.

- Dem. prin inducție după muchiile adăugate în AMA

- P_1: (u, v), $w(u, v) = \min\{w(u', v') \mid (u', v') \in E\}$

  $\Rightarrow (u, v) \in Arb$

- $P_n \Rightarrow P_{n+1}$:

  - $Arb(u) != Arb(v) \Rightarrow (u, v)$ muchie cu un capăt în $Arb(u)$
  - (u, v) are cel mai mic cost din muchiile cu un capăt în u

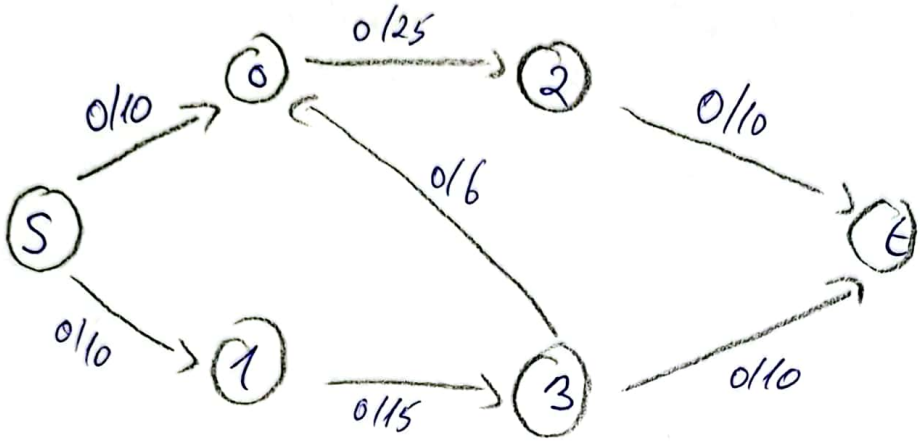
  $\Rightarrow (u, v) \in Arb$

Ford - Fulkerson $O(E \cdot flux_{maxim})$

- calculul fluxului maxim

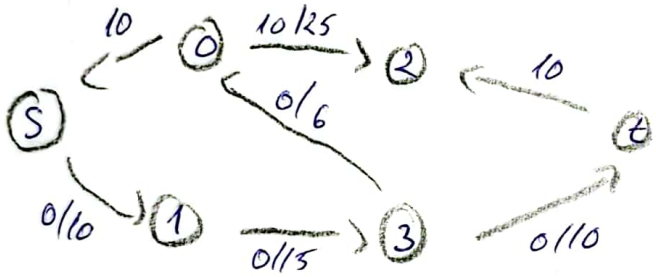
Edmonds - Karp $O(VE^2)$

- când se ajunge la x/x se întoarce muchia



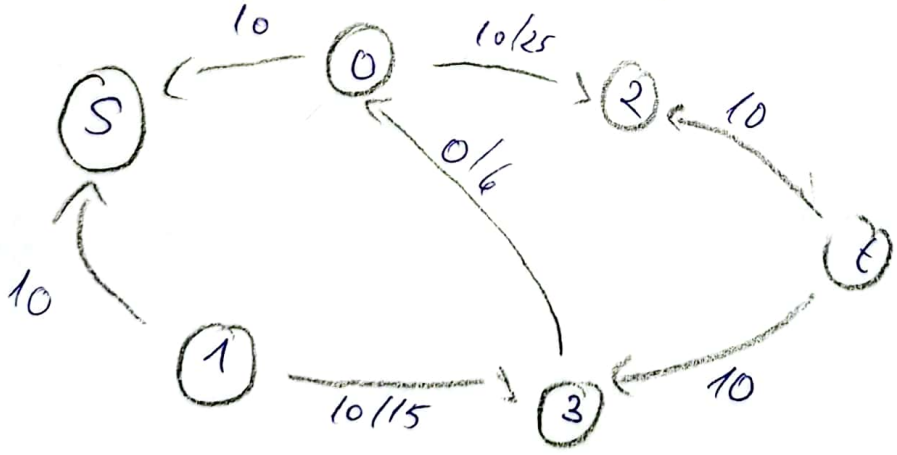
Pas 1

- cale reziduală $p = S \to 0 \to 2 \to t$
- minimul este 10 $\Rightarrow c_{f_1} = 10 \Rightarrow |f| = 10$



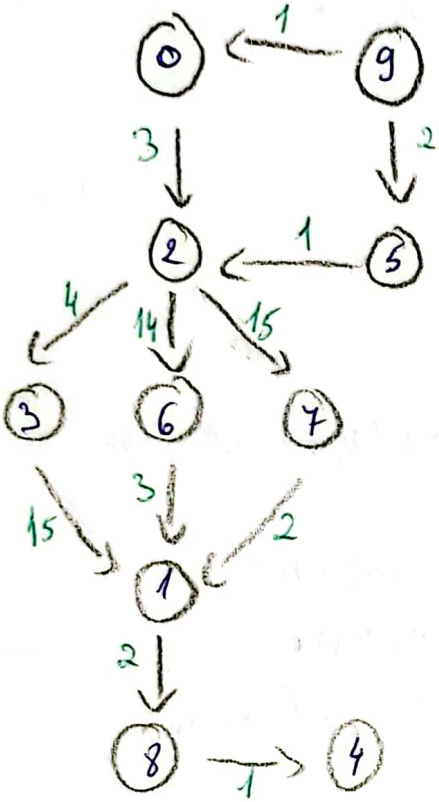
Pas 2

- cale reziduală $p = S \to 1 \to 3 \to t$
- minimul este 10 $\Rightarrow c_{f_2} = 10 \Rightarrow |f| = 20$



Algoritm euristic A*

- $h(n)$: costul estimat al drumului optim de la n la nodul soluție
- $h^*(n)$ = costul exact al porțiunii de drum optim $n \dots Y$
- $f(n) = h(n) + g(n)$
- Complexitate - liniară $|h^*(n) - h(n)| \leq \delta$
  subexponențială $|h^*(n) - h(n)| \leq O(\log(h^*(n)))$
  exponențială, altfel



- sursa $s=9$ destinație $t=4$
- $h(x) = \{ \min(cost(x,x')) \mid (x,x') \in E \}, x != t$
- $h(t) = h(4) = 0$
- $g(x)$ - distanța minimă de la sursă la fiecare nod din OPEN, pe calea cu nodurile din CLOSED

Noduri	h
0	3
1	2
2	4
3	15
4	0
5	1
6	3
7	2
8	1
9	1

$f = g(n) + h(n)$

- la fiecare pas se face diagrama cu vecinii nodului din select
- se selectează nodul cu f min

OPEN	9
f	1
g	0

CLOSED = { }
select (9)

OPEN	0	5
f	4	3
g	1	2

CLOSED = {9}
select (5)

OPEN	2
f	7
g	3

CLOSED = {9, 5}
select (2)

OPEN	3	6	7
f	22	20	20
g	7	17	18

CLOSED = {9, 5, 2}
select (6)

OPEN	1
f	22
g	20

CLOSED = {9, 5, 2, 6}
select (1)

OPEN	8
f	23
g	22

CLOSED = {9, 5, 2, 6, 8}
select (8)

OPEN	4
f	23
g	23

CLOSED = {9, 5, 2, 6, 1, 8, 4}
cost = 23

• Euristici - consistență și monotonie

Fie h o euristică

- h-consistentă $\Leftrightarrow \forall$ noduri n, n', a.i n' accesibil din n

$$h(n) \leq h(n') + h_2(n, n')$$

'costul unui drum optim de la n la n'

- h-monotonă $\Leftrightarrow \forall$ noduri n, n' a.i n'-succesorul lui n

$$h(n) \leq h(n') + c(n, n')$$

'costul arcului (n, n')

• h consistentă $\Rightarrow$ h monotonă

a) h-cons $\Rightarrow$ h mon

$n' \in succs(n) \Rightarrow h_2(n, n') = c(n, n') \Rightarrow h(n) \leq h(n') + c(n, n') \Rightarrow h$-mon

b) h-mon $\Rightarrow$ h cons

Fie $n = n_1, n_2, \ldots, n_q = n'$ - drum optim $n \ldots n'$ cu cost $h_2(n, n')$

$$\Rightarrow h(n) = h(n_1) \leq h(n_2) + c(n_1, n_2) \leq h(n_3) + c(n_1, n_2) + c(n_2, n_3) \leq \ldots$$
$$\leq h(n_q) + c(n_1, n_2) + \ldots + c(n_{q-1}, n_q) = h(n_q) + h_2(n_1, n_q)$$

$$\Rightarrow h(n_1) \leq h(n') + h_2(n, n') \Rightarrow h\text{-cons}.$$

• h-consistentă $\Rightarrow$ h-admisibilă

h-cons $\Leftrightarrow h(n) \leq h(n') + h_2(n, n')$, $\forall$ n' accesibil din n

Fie $n' = \gamma \in \Gamma \Rightarrow h_2(n, \gamma) = \min\{h_2(n, \gamma') \mid \gamma' \in \Gamma\} = h^*(n)$

$$\Rightarrow h(n) \leq h(\gamma) + h^*(n), \; h(\gamma) = 0 \Rightarrow h(n) \leq h^*(n)$$

$\Downarrow$

euristică admisibilă

Algoritmi pentru jocuri.
Minimax $\alpha-\beta$

- Când sunt 3 jucători, fiecare face alegerea care îi maximizează câștigul propriu.
- α = MAX
- β = MIN
- $\alpha \geq \beta$ => se taie ramura
- Inițial $\alpha = -\infty$ $\beta = +\infty$

